

Claw-SWE-Bench: A Benchmark for Evaluating OpenClaw-style Agent Harnesses on Coding Tasks

ABSTRACT

General-purpose agents such as OpenClaw are increasingly used as autonomous tool users, but their coding ability is difficult to measure under SWE-bench: a generic agent does not by itself satisfy the clean Docker workspace, patch, and prediction contract required for scoring. We introduce CLAW-SWE-BENCH, a multilingual SWE-bench-style benchmark and adapter protocol that makes heterogeneous agent harnesses, or claws, comparable under fair settings including a fixed prompt, runtime budget, workspace contract, patch extraction procedure, and evaluator. The full benchmark contains 350 GitHub issue-resolution instances across 8 languages and 43 repositories, drawn from SWE-bench-Multilingual and SWE-bench-Verified-Mini after future-commit cleanup. We also release CLAW-SWE-BENCH LITE for faster validation, which is an 80-instance subset selected by a cost-aware, rank-aware procedure over 17 calibration columns. On the full benchmark, OpenClaw with a minimal direct-diff adapter scores only 19.1% Pass@1, whereas the full adapter reaches 73.4% with the same GLM 5.1 backbone, showing that adapter design is essential for enabling OpenClaw-style harnesses to perform coding tasks effectively. Across an OpenClaw $\times$ nine-model sweep and a five-claw $\times$ two-model sweep, model choice changes Pass@1 by 29.4 pp and harness choice by 27.4 pp under fixed models; systems with similar accuracy can differ substantially in total API cost. Claw-SWE-Bench therefore treats harness and cost accounting as first-class axes of SWE-style coding-agent evaluation, providing both a full benchmark and a low-cost reference set for reproducible comparison. The data is available at <https://github.com/opensquilla/claw-swe-bench> and <https://huggingface.co/datasets/TokenRhythm/Claw-SWE-Bench>.

1 Introduction

General-purpose agents exemplified by OpenClaw [26] have rapidly expanded into productivity tools, browser automation, computer-use tasks, and scientific assistance. Yet it remains unclear whether such agents can serve as effective coding agents on real software-engineering tasks. Existing public evaluations mostly cover open-ended productivity tasks, workplace collaboration tasks [10, 20, 36], or broad agent leaderboards [6, 18, 27, 34]; direct evidence about their repository-level coding ability is still limited.

The natural way to test this ability is to use a SWE-bench-style benchmark [16], because SWE-bench has become the de facto standard for repository-level coding agents. However, leading SWE-bench-style reports often package the prompt template, agent loop, tool interface,

remains a caveat rather than a controlled measurement. The unresolved challenge is that no SWE-bench-style benchmark has made the agent harness a controlled experimental variable.

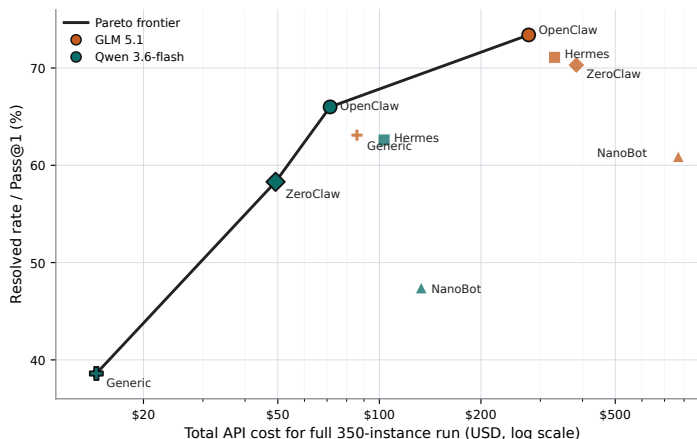


FIGURE 1 **Resolve-rate-cost Pareto frontier.** Data are from the five-claw $\times$ two-model sweep in Table 3. Each point is one claw-model combination on the full 350-instance evaluation; the vertical axis is Pass@1 / resolved rate, and the horizontal axis is full-run total API cost (USD, log scale). The black line connects non-dominated operating points.

This conflation also hides resource cost. A real coding agent is not a single model call: it repeatedly reads files, edits code, runs commands, and waits for remote model responses. The same Pass@1 can correspond to very different token usage, wall-clock duration, and interaction length. Reporting only resolved rate rewards systems that rely on longer exploration or higher budgets, and can lead to misinterpreting systems that are cheaper or faster but more brittle. A coding-agent benchmark therefore needs to report accuracy together with end-to-end cost under a fixed outer budget. Cost determines whether a full evaluation, regression test, or system iteration is actually affordable, and affects whether small teams and academic groups can participate in such benchmarking.

Figure 1 illustrates this point using the full 350-instance sweep over five claws and two models. Each point is one claw-model combination under the same evaluation protocol, with Pass@1 on the vertical axis and total API cost on the horizontal axis; the black curve marks the Pareto frontier, where no other combination is both cheaper and more accurate. Accuracy and cost do not move in lockstep. We therefore treat cost-aware reporting as part of the benchmark design rather than an auxiliary log appended after resolved rate.

We introduce *Claw-SWE-Bench*, a multilingual SWE-bench-style benchmark that treats the agent harness as a controlled experimental variable. The benchmark decomposes the evaluation stack into a fixed base – prompt template, task set, execution container, per-instance timeout, patch extraction, and evaluator – plus a replaceable harness slot. Harnesses enter this slot through a shared adapter protocol exposing a small set of lifecycle methods (the full interface is described in §2.2). The workload contains 350 real GitHub issue-resolution instances across 8 programming languages and 43 repositories, drawn from SWE-bench-Multilingual [31] and SWE-bench-Verified-Mini [15], and evaluated with the upstream SWE-bench evaluator. All systems share the same outer budget and report total API cost, average wall-clock duration,

and cache hit rate alongside Pass@1, so accuracy and end-to-end cost can be interpreted in the same table and on the same Pareto plane.

To lower the barrier to use, we also release *Claw-SWE-Bench Lite*, an 80-instance low-cost subset for users who need to evaluate model coding ability or iterate on harness design without repeatedly paying for the full 350-instance, multi-harness $\times$ multi-model grid. Lite is not a convenient showcase sample; it is designed to preserve the scale, language distribution, key rankings, and cost structure of the full set under limited budget, enabling shorter feedback loops for model replacement, adapter debugging, prompt adjustment, and regression testing. Lite uses the cost-aware, rank-aware selection method in §3.2, optimizing resolve-rate parity, pairwise ranking stability, and cost parity over 17 calibration columns. The final 80-instance Lite subset reduces full-run cost to about 22.9% of full-350; over the 17 calibration columns, the mean Pass@1 values on full-350 and Lite-80 are 0.639 and 0.643, a difference of about 0.4 pp. A K-sweep shows that the minimum acceptable per-language size falls in $K^* \in [8, 10]$; we release the conservative and stable $K=10$ point. Lite does not replace the full benchmark, but provides a practical entry point for screening, regression evaluation, and result checking under constrained budgets.

Using this protocol and Lite subset, CLAW-SWE-BENCH provides a common task set, budget, and scoring pipeline for measuring differences in harness coding ability and run cost under comparable conditions. We conduct two complementary studies: a model sweep that fixes OPENCLAW and evaluates nine LLMs, and a claw sweep that fixes two representative models (GLM 5.1 and Qwen 3.6-flash) and evaluates five claws. First, a general-purpose OpenClaw harness achieves competitive Pass@1 on real issue-resolution tasks, showing that a general harness can enter SWE-bench-style coding evaluation through an adapter. Second, harness choice is a first-order factor: under a fixed model, the claw spread reaches 12.5 pp on GLM 5.1 and 27.4 pp on Qwen 3.6-flash, large enough to reorder leaderboard conclusions if the harness is not specified. Finally, accuracy and cost are not simply aligned; comparable SWE-style results require explicit control and disclosure of harness, budget, cost metric, and cache accounting.

2 Claw-SWE-Bench

The first question in this paper is whether a general-purpose agent such as OpenClaw can enter a SWE-bench-style evaluation of real coding tasks. To make this question experimentally testable, we first specify the SWE-bench [16] scoring contract. Given the `problem_statement`, target `repo`, and `base_commit` for a real GitHub issue, a system must submit a diff patch that can be applied to the repository checkout. The official evaluation harness does not read an interaction trace or a final natural-language answer. It reads a prediction file in which each instance contains at least `instance_id`, `model_name_or_path`, and a string-valued `model_patch`. The evaluator then prepares the repository in the Docker evaluation environment for that instance, applies the patch to the checkout under `/testbed`, and runs repository-level tests to determine whether the instance is resolved. In short, the core SWE-bench interface is an evaluator-facing patch prediction, not a generic agent session.

Coding harnesses such as SWE-agent [30] are designed around this contract. OpenClaw, by

contrast, is normally run as a more general agent interaction and therefore cannot be treated as a SWE-bench evaluation target without adaptation. First, the SWE-bench Docker image is primarily a reproducible target-repository, dependency, and test environment; it does not itself provide the agent lifecycle, tool configuration, API access, session state, or workspace management required by OpenClaw. These runtime dependencies and state must be brought inside a controlled container boundary while ensuring that the agent’s actual code edits occur in `/testbed`. Second, general-purpose agents often signal completion through final text, structured messages, or internal logs, whereas the SWE-bench evaluator reads only the `model_patch` field. Explanatory answers are not directly scorable. Third, a general agent can create session files, metadata, caches, or other non-solution artifacts during execution; if these enter `git diff`, they contaminate the patch submitted to the evaluator.

These limitations do not imply that OpenClaw lacks coding ability. They imply that native OpenClaw cannot directly enter the SWE-bench scoring pipeline. The premise we first challenge is that SWE-bench-style coding tasks must be solved only by purpose-built coding harnesses. General-purpose agents can participate in real issue resolution if an adapter constrains their behavior to concrete repository edits and converts the final repository state into an evaluator-readable patch. Once this access problem is solved, the next step is to define a unified evaluation standard that compares the coding ability and run cost of different claws or harnesses under the same tasks, budgets, and scoring pipeline.

We therefore propose CLAW-SWE-BENCH, a multilingual SWE-style benchmark and execution protocol for evaluating coding-agent harnesses. It combines 350 real GitHub issue-resolution tasks across 8 programming languages with a unified adapter layer, allowing heterogeneous “claws” – agent harnesses that wrap LLMs into autonomous coding systems – to run under the same evaluation protocol.

CLAW-SWE-BENCH achieves this in two layers. The first layer is the adapter: it connects the native execution style of a general or specialized harness to the repository-editing and patch-prediction process required by SWE-bench, making these systems eligible for the same class of coding tasks. The second layer is a shared orchestrator: it fixes the task set, repository state, task prompt, Docker runtime, outer budget, patch extraction, prediction format, and downstream SWE-bench evaluation, elevating the harness from an incidental implementation detail to an experimental variable. Under this control, differences in Pass@1, wall-clock duration, and turn traces can be attributed to model or harness dimensions rather than to inconsistent evaluation protocols. The rest of this section describes the workload source, adapter protocol, and standardized execution pipeline.

2.1 Workload Source and Composition

The full CLAW-SWE-BENCH workload is built from two upstream SWE-bench-derived sources. SWE-bench-Multilingual [31] contributes 300 non-Python instances covering Java, Go, Rust, JavaScript/TypeScript, C/C++, Ruby, and PHP. SWE-bench-Verified-Mini [15] contributes 50 human-validated Python instances. Together, the full benchmark contains 350 real GitHub issue-resolution tasks across 8 programming languages and 43 repositories.

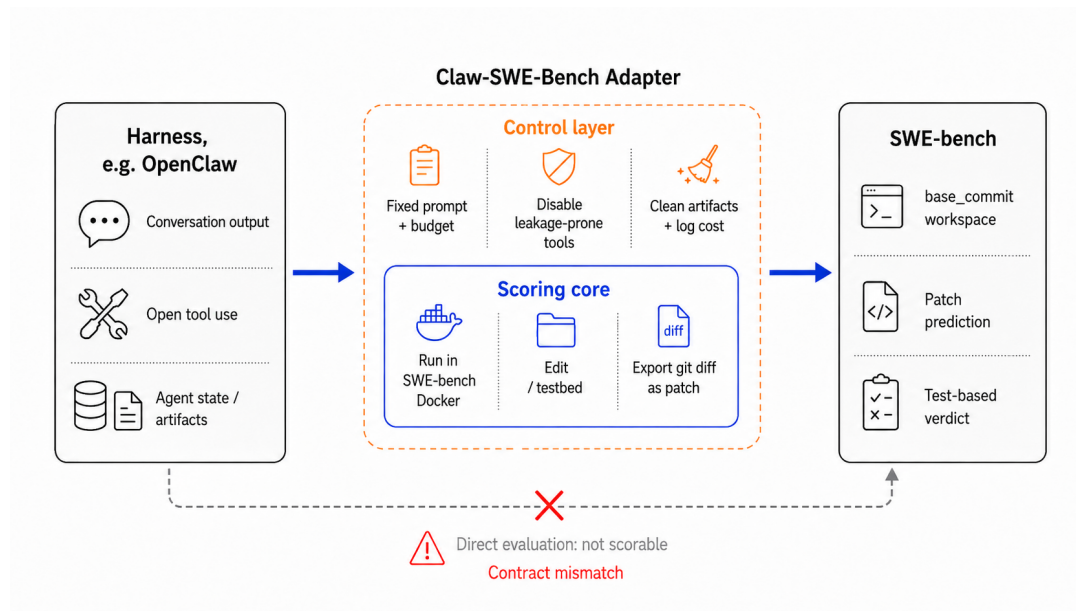


FIGURE 2 Contract mismatch between OpenClaw-style harnesses and SWE-bench. The adapter converts a general agent interaction into a SWE-bench-scored patch prediction, while outer controls ensure fairness, comparability, and traceable cost.

Each instance preserves the upstream SWE-bench task format and evaluation assets, including `problem_statement`, `repo`, `base_commit`, the corresponding Docker evaluation image, and the repository-level tests used for scoring. This combination serves two purposes. First, the benchmark remains compatible with SWE-bench’s patch-based evaluation. Second, multilingual tasks and human-validated Python tasks jointly provide broader real-software-engineering coverage, so harness comparisons are not limited to one language or one upstream subset. All model–harness combinations are run on the same 350 instances, allowing resolved-rate and cost differences to be interpreted under a fixed workload.

2.2 Adapter Protocol

The adapter protocol is the first layer of CLAW-SWE-BENCH. It does not require different harnesses to use the same internal agent loop; instead, it standardizes the interface between a harness and the benchmark lifecycle. Each supported harness implements the same abstract methods: `create_agent`, `send_task`, `backup_session`, `delete_agent`, and `get_docker_args`. The shared orchestrator drives a run only through these methods, without needing to know which harness is underneath. This design decouples the benchmark lifecycle from agent implementation: container management, prompt instantiation, patch collection, prediction writing, metadata recording, resume support, and evaluation are implemented by the benchmark layer, while each harness adapter only connects its agent to that lifecycle and provides the code needed to drive the agent inside the container.

At runtime, the shared orchestrator enforces the access boundary shown in Figure 2. Container startup, repository reset, prompt instantiation, patch collection, prediction writing, metadata

recording, and evaluation are handled uniformly by the benchmark layer. The adapter provides harness-specific hooks to create or configure the agent, dispatch the instantiated task, save run artifacts, and clean harness state. This boundary is deliberate: the benchmark layer owns the task-facing environment and evaluator-facing patch format, while the internal agent loop remains part of the harness being studied.

Crucially, candidate patches are collected from repository state rather than parsed from an agent’s final message. An agent expresses a solution only by editing files in the repository. This makes the output contract independent of whether the harness natively produces JSON, plain text, a final narrative response, or no structured response at all.

All harnesses are launched through the same command-line entry point, `run_infer.py`. The evaluator specifies the harness name, dataset configuration, model identifier, run identifier, timeout, worker count, and optional instance filters. Dataset metadata is loaded from configured SWE-bench sources, and each instance is represented by the fields required by the protocol: `instance_id`, `repo`, `base_commit`, and `problem_statement`. A harness registry maps string IDs (`openclaw`, `hermes`, `nanobot`, `zeroclaw`, and `generic`) to adapter classes. Adding a new claw only requires implementing the adapter interface and registering it in the harness map; the dataset loader, Docker workspace manager, prompt builder, patch collector, prediction writer, and evaluator remain unchanged.

2.3 Standardized Execution Pipeline

The adapter determines whether heterogeneous harnesses can enter a common evaluation protocol. Outside that boundary, CLAW-SWE-BENCH further fixes the evaluation-stack components that would otherwise confound harness comparisons.

Runtime and workspace. Each task runs inside its corresponding SWE-bench evaluation Docker image, with the repository reset to the instance’s `base_commit` and mounted at `/testbed`. For the seven non-Python languages from SWE-bench-Multilingual, we also handle future-commit visibility during workspace preparation. While inspecting the containers, we found that some images still exposed Git commits after `base_commit`; if left unchanged, an agent could inspect future fixes through `git log` or `git show`, which is incompatible with the patch-based evaluation contract. The runner therefore removes reachable future commits so that the agent can only read, edit, and run code within the history boundary of the issue. All harnesses share the same outer budget: a 3600-second wall-clock timeout, one run per instance, and fixed worker concurrency. Harness-specific dependencies can be supplied through Docker arguments or bind mounts, but the repository state, evaluation image, and outer budget perceived by the agent are fixed. These budget controls prevent longer exploration time from being mistaken for stronger harness design and make cost metrics comparable across harnesses. Because different harnesses define a “turn” differently, wall-clock duration is the primary comparable resource metric; turn count is treated as a diagnostic trace. Token statistics are available for some harnesses but not exposed uniformly by all systems, so they are not the sole cross-harness metric.

Prompt instantiation. Every instance is instantiated from the same task-prompt template. The prompt includes the problem statement and base commit, instructs the agent to work in

`/testbed`, forbids `git add` and `git commit`, and asks the agent not to modify test files. Thus the task-facing input message is held fixed across harnesses. The protocol does not attempt to standardize a harness’s internal system prompt, tool schema, parser hints, memory strategy, or stopping rule; these remain part of harness design and therefore part of the experimental variable.

Patch and scoring contract. Candidate solutions are collected from repository state rather than parsed from the agent’s final response. After a harness terminates, times out, or returns an error, the runner computes the diff against the base commit, removes known non-solution artifacts, and writes a SWE-bench-compatible prediction. This centralized patch-submission process allows heterogeneous harnesses to be compared even when their native outputs differ: JSON, plain text, natural-language summaries, and missing structured responses are all reduced to the same evaluator-facing patch format. Evaluation is then performed by the official SWE-bench harness.

To separate “placing OpenClaw inside Docker” from “reliably satisfying the SWE-bench scoring contract,” we also define a minimal *bare adapter* as a diagnostic baseline. The bare adapter provides only minimal integration: it enters the corresponding Docker workspace for each instance, sends the issue description to OpenClaw, and disables network retrieval that would clearly violate fairness. It does not perform full workspace alignment, future-commit cleanup, shared phase prompting, Git-based patch extraction, or patch cleaning; instead, it asks the model to output a unified diff directly in the final response. By contrast, the full adapter used in the main experiments requires the agent to edit files under `/testbed`, after which the runner exports `model_patch` from the final repository state. This comparison tests the necessity of the adapter, not the attribution of individual adapter components.

3 Claw-SWE-Bench Lite

The full 350-instance benchmark is the standard evaluation surface in this paper, but it is not suitable as the feedback loop for every development iteration. A full-350 run requires substantial token usage, API cost, wall-clock time, and log inspection effort. During adapter debugging, prompt modification, model replacement, or regression testing, repeatedly running the full set can make evaluation itself the bottleneck. CLAW-SWE-BENCH LITE is therefore designed as a low-cost companion to the full benchmark rather than as a replacement leaderboard: with 80 instances, it approximates the Pass@1 scale, per-language distribution, cross-claw relative behavior, and run-cost structure of full-350, allowing researchers to triage system changes with a shorter feedback loop before returning to full-350 for final reporting.

3.1 Lite Subset Definition

Lite-80 selects 10 instances from each of the 8 languages in full-350. The 70 non-Python instances come from SWE-bench-Multilingual, and the 10 Python instances come from SWE-bench-Verified-Mini [15], matching the source of the Python portion of the full set. In addition to language balance, Lite enforces a fixed within-language difficulty-quartile quota of 2/3/3/2 over

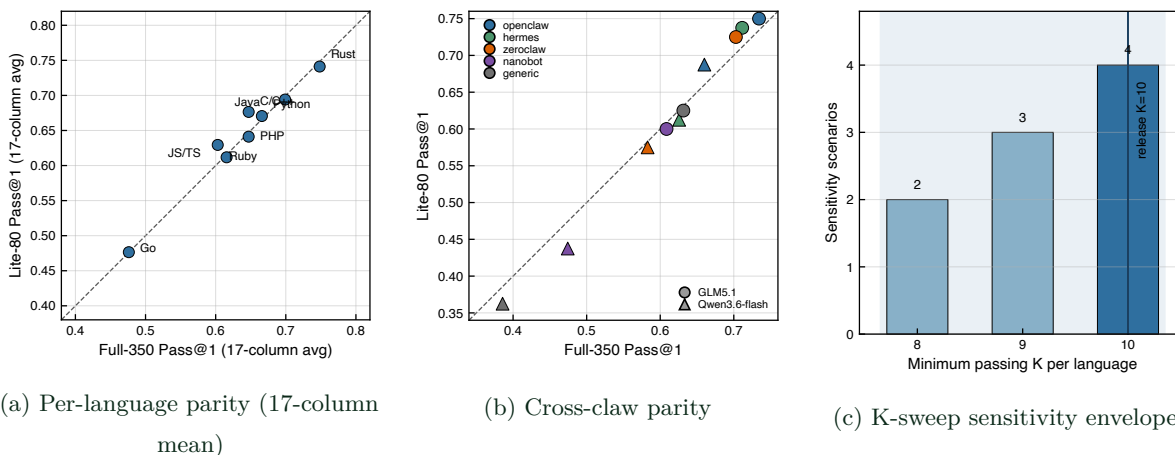


FIGURE 3 Lite-80 parity with full-350. (a) Per-language comparison between full-350 and Lite-80 Pass@1, averaged uniformly over the 17 calibration columns. (b) Cross-claw Pass@1 comparison between full-350 and Lite-80 over 5 claws $\times$ 2 shared models. (c) K-sweep sensitivity envelope; the minimum acceptable K falls in $[8, 10]$ across scenarios, and the release uses the conservative stable point $K=10$, or 10 instances per language.

$Q_1/Q_2/Q_3/Q_4$, avoiding implicit resampling of any language toward unusually easy or unusually hard tasks. The final subset covers 34 of the 43 repositories in full-350 (79%), preserving a substantial amount of repository diversity.

Lite is not a simple random sample. It is fitted to full-350 behavior over 17 calibration columns. These columns include 9 OpenClaw model columns and 8 cross-claw columns from 4 non-openclaw claws (hermes, nanobot, zeroclaw, and generic) evaluated on two shared models, GLM 5.1 and Qwen 3.6-flash. This calibration pool spans both model variation and claw variation. Lite’s objective therefore goes beyond preserving an average resolved rate: it aims to preserve the comparability scale of different systems on the full benchmark.

3.2 Cost-Aware, Rank-Aware Selection

We formulate Lite selection as a binary selection problem over the 350 full-set instances. The variable $x_i \in \{0, 1\}$ indicates whether instance i is included in Lite. Hard constraints require selecting 10 instances per language and satisfying the fixed 2/3/3/2 difficulty-quartile quota within that language. Difficulty quartiles are computed from the mean resolved rate over the calibration pool, so they reflect relative difficulty under multiple models and claws rather than under a single system.

The objective controls three sources of bias. The first term is resolve-rate parity: over the 17×8 grid of calibration columns by language, it minimizes the L1 difference between the Lite-estimated rate and the true full-350 rate. The second term is a pairwise ranking hinge: when two calibration columns differ by more than $\text{RANK_EPS} = 0.03$ on full-350, a penalty is applied if Lite reverses the order or falls within a 0.05 margin ($\lambda = 1.0$). The third term is cost parity: for each calibration column, it minimizes the log-cost discrepancy between Lite and full-350 ($\text{cost_alpha} = 1$), preventing the subset from matching resolved rate while being biased

toward unusually cheap or expensive instances. Optimization uses per-language 200-restart within-quartile 1-swap local search, which keeps all hard constraints satisfied throughout the search and avoids reliance on an external solver.

3.3 Validation Results and the 80-Instance Scale

Figure 3 summarizes the main validation results for Lite-80. Across the 17 calibration columns, mean Pass@1 is 0.639 on full-350 and 0.643 on Lite-80, a difference of about +0.4 pp. Per-language deviations are small overall: Go, JS/TS, PHP, and Python are all within 1 pp; the two largest deviations are C/C++ (+2.94 pp) and Ruby (+2.65 pp). In the 5 claws $\times$ 2 models cross-claw check, which is closer to how leaderboards are used, the mean absolute Lite–full difference is 1.88 pp and the maximum difference is 3.68 pp (nanobot $\times$ Qwen 3.6-flash). These results indicate that Lite-80 does not merely fit one local OpenClaw model, but preserves a cross-model and cross-claw evaluation scale.

The cost side must also be checked. Lite-80’s actual per-instance cost is close to that of full-350; because the number of instances falls from 350 to 80, a full Lite run costs about 22.9% of a full run. Broken down by resource type, the full-run ratios for input tokens, output tokens, cache-read tokens, and wall-clock duration are approximately 22.2%, 23.6%, 22.6%, and 23.0%, respectively. Lite therefore provides an evaluation surface at roughly one quarter of the cost, rather than lowering cost by selecting anomalously cheap examples.

The choice of 80 instances also comes from an explicit K -sweep rather than a convenient round number. We scan subset size in units of K instances per language and repeat selection and validation across different margin, restart, seed, and mirror-parity scenarios. Sensitivity analysis finds that the minimum acceptable size lies in $K^* \in [8, 10]$: two scenarios pass at $K=8$, three require $K=9$, and four structural-perturbation scenarios require $K=10$. We release $K_{\max}^* = 10$, or 8 languages $\times$ 10 instances = 80 instances. At this size, the resolve gates (R-A/R-B/R-C), cost gates (C-A/C-B/C-C), and operational composite gate all pass. Lite-80 is therefore the smallest conservative stable release point under the sensitivity envelope: smaller K values can work in some configurations, but are not robust enough to serve as the default reusable low-cost benchmark.

4 Experimental Setup

We use CLAW-SWE-BENCH to study two sources of variation in SWE-style coding-agent evaluation: the LLM, and the claw that wraps the LLM into an autonomous coding system. We report two complementary experimental grids rather than an exhaustive claw $\times$ model grid over all 350 instances. First, we fix a reference claw and sweep the model axis. Second, we fix two representative models and sweep the claw axis. Finally, we validate whether the Lite subset preserves the trend of the full set.

Claws. We evaluate five claws: OPENCLAW [26], HERMES-AGENT [23], ZEROCLAW [37], NANOBOT [14], and a GENERICAGENT [19]. In this paper, a claw is the harness-specific agent loop running inside the standardized CLAW-SWE-BENCH protocol. All claws receive the same

task prompt, run in the same SWE-bench Docker workspace, and obey the same outer budget.

Models. The model sweep uses OPENCLAW with nine LLMs spanning a broad capability and cost range: GPT 5.5 [24], Claude Opus 4.7 [2], GLM 5.1 [35], DeepSeek-V4 Pro [8], DeepSeek-V4 Flash [7], Kimi 2.6 [22], Qwen 3.6-flash [1], MiniMax M2.7 [21], and Seed 2.0-mini [4]. The claw sweep uses two representative models: GLM 5.1, a stronger mid-tier model, and QWEN 3.6-FLASH, a lower-cost small model. This two-model claw sweep exposes both high-capability behavior, where ceiling effects may reduce visible claw differences, and small-model behavior, where harness brittleness and stopping policy often matter more. Model inference is routed through external API providers; provider mappings and model identifiers are listed in the reproducibility appendix.

Evaluation metrics. The primary metric is PASS@1, defined as the fraction of instances whose submitted patch is marked RESOLVED by the SWE-bench evaluator:

$$\text{PASS@1} = \frac{\text{\#RESOLVED}}{\text{\#INSTANCES}}.$$

In addition to accuracy, we report two classes of efficiency metrics. The first class is end-to-end run cost, including TOTAL COST (USD) for the full 350-instance run and mean wall-clock duration. TOTAL COST comes from the corresponding API provider or cache-proxy billing logs and measures the actual resource cost of a full evaluation; duration is recorded by the outer runner and includes remote API latency. The second class is cache-use diagnostics. We report CACHE HIT RATE:

$$\text{CACHEHIT} = \frac{\text{\#CACHEREADTOKENS}}{\text{\#INPUTTOKENS} + \text{\#CACHEREADTOKENS}}.$$

Cache hit rate affects actual API cost and should therefore be disclosed with cost, but it is not a coding-capability metric: it depends on provider cache policy, adapter call paths, and context-reuse strategy.

Lite held-out validation. In addition to the full-350 main experiments, we use OPENSQUILLA as a held-out system to check whether Lite-80 reproduces the aggregate evaluation scale of the full benchmark. OPENSQUILLA is not used to construct or calibrate the Lite subset. The experiment only compares OPENSQUILLA’s PASS@1 on Lite-80 and full-350. Both runs use the same adapter protocol, outer budget, and SWE-bench evaluator, and we measure approximation quality by the percentage-point gap between the Lite-80 rate and the full-350 rate.

Runtime configuration. All experiments use the same outer runtime configuration. Each instance runs in its SWE-bench evaluation image, with the repository checkout located at /testbed. The instantiated task prompt, patch collector, evaluator, and aggregation code are shared across all claws and models. A per-instance wall-clock timeout of 3600 seconds, one run per instance, and worker concurrency fixed at 3. Experiments run on a 16-core CPU server with 61 GiB of memory and no local GPU; all model inference is performed through remote APIs.

Adapter diagnostic. Beyond the main experiments, we run a bare-vs-full adapter diagnostic with GLM 5.1. Both conditions use the same full-350 workload and SWE-bench evaluator. The bare adapter provides only minimal Docker access and fairness restrictions, and asks the model to output a unified diff directly. The full adapter uses workspace preparation, the shared prompt,

Configuration	Resolved	Pass@1	Apply Failed
Bare adapter	67/350	19.1	69.1%
Full adapter	257/350	73.4	< 1.5%

TABLE 1 Diagnostic comparison between the bare adapter and the full adapter. Both use the same GLM 5.1 backbone and the full-350 workload; the bare adapter is a minimal directly scorable baseline, not a component ablation of the full adapter. **Apply Failed** is the fraction of instances whose submitted patch cannot be applied to the repository by the SWE-bench evaluator.

Git-based patch extraction, and patch cleaning from our protocol. This diagnostic quantifies the effect of the complete adapter on scorable evaluation, and should not be interpreted as a single-component ablation.

Leak-fix evaluation protocol. The main experiments use results after cleaning future-commit visibility. Specifically, for the seven non-Python SWE-bench-Multilingual task languages, each instance preparation removes reachable Git history later than `base_commit` and then runs under the same adapter protocol. The Python portion comes from SWE-bench-Verified-Mini and is not affected by this Multilingual container issue. Except for the before/after cleanup comparison reported in §5.3, all Multilingual results in the following tables and figures use the cleanup setting.

5 Results

Except for the Lite held-out validation, all main results below report single-run aggregates on the full 350 instances, with worker concurrency fixed at 3. Unlike SWE-bench-style tables that report only resolved rate, we also report TOTAL COST, mean wall-clock duration, token usage, turn count, and CACHE HIT RATE, so coding ability and practical evaluation cost can be interpreted in the same coordinate system. For OpenClaw × GLM 5.1, we use the cost and cache accounting from the 9-model leak-fix result table; for OpenClaw × Qwen 3.6-flash, we use the cache-fixed 5-claw cross table and add the mean turn count.

5.1 The Adapter Makes a General Agent Scorable

We first test whether the adapter is merely an engineering wrapper or a necessary condition for OpenClaw to be reliably scored by SWE-bench. Table 1 compares the same GLM 5.1 backbone under the bare adapter and the full adapter. The bare adapter can place OpenClaw in the SWE-bench Docker environment and send the task, but still asks the model to write a unified diff directly in its final response. The full adapter instead lets the model edit repository files through tools and has the runner export the patch from Git state.

The results show that minimal access is insufficient to create a reliable SWE-bench evaluation target. The bare adapter reaches only 19.1% resolved rate. The main bottleneck is not that the model cannot edit code at all, but the fragility of directly generating unified-diff text: line numbers, context, hunk headers, or trailing newlines can make the patch fail to apply. The

Model	Type	Resolved	Pass@1	Cost	Dur	In(M)	Out(M)	Turns	Cache
GPT 5.5	Flagship	273	78.0	1399.1	603.7	40.3	15.7	67.0	97.3
Claude Opus 4.7	Flagship	270	77.1	1082.0	424.6	35.6	6.2	61.6	97.0
GLM 5.1	Flagship	257	73.4	277.0	586.8	27.6	9.3	80.6	96.5
DeepSeek-V4 Pro	Flagship	251	71.7	81.3	662.3	19.3	11.0	47.1	97.4
Kimi K2.6	Flagship	234	66.9	633.7	1235.3	75.6	12.1	78.7	92.1
MiniMax M2.7	Flagship	215	61.4	196.7	1165.6	25.0	9.0	94.8	96.2
DeepSeek-V4 Flash	Flash	246	70.3	8.2	430.0	13.8	12.9	51.2	98.5
Qwen 3.6-flash	Flash	231	66.0	71.5	636.0	38.9	7.5	87.9	97.6
Seed 2.0-mini	Flash	170	48.6	19.4	1153.0	89.8	21.6	44.4	79.4

TABLE 2 LLM-axis variation: OpenClaw $\times$ 9 models on the full 350-instance Claw-SWE-Bench. Cost is total API cost for the full run (USD); In/Out are total input/output tokens (millions); Turns is average turns; Cache is cache hit rate. Rows are sorted by Pass@1; the best Pass@1 and lowest Cost are in **bold**.

full adapter shifts the output responsibility from “the model writes patch text” to “the model edits repository files and the runner exports the patch,” reducing apply failures below 1.5% and raising resolved rate to 73.4%. The following experiments therefore measure model and claw differences under a unified scoring contract, rather than testing whether a native agent can hand-write a SWE-bench-compatible diff.

5.2 Variation Along the LLM Axis

To isolate the contribution of the LLM, we fix OpenClaw as the reference claw and sweep nine models on the full 350-instance set. Table 2 reports aggregate results. The highest resolved rate is achieved by GPT 5.5, at 78.0% (273/350), followed by Claude Opus 4.7 at 77.1% (270/350). The lowest cell is Seed 2.0-mini, at 48.6% (170/350). Thus, under the same OpenClaw scaffold, changing only the model produces a 29.4 pp Pass@1 spread, confirming that model choice remains a major source of coding-agent performance.

Accuracy ranking, however, is not cost ranking. GPT 5.5 has the highest Pass@1, but its full 350-instance run costs \$1399.1; Claude Opus 4.7 is only 0.9 pp lower, with cost \$1082.0. By contrast, DeepSeek-V4 Pro reaches 71.7% Pass@1 at total cost \$81.3, while DeepSeek-V4 Flash reaches 70.3% at only \$8.2. Qwen 3.6-flash reaches 66.0% Pass@1 at \$71.5; GLM 5.1 reaches 73.4% under cache-fixed cost accounting at \$277.0. These results show that cost-aware reporting is not an auxiliary log but a necessary dimension for interpreting benchmark results: similar resolved rates can correspond to evaluation costs that differ by orders of magnitude.

Cache hit rate explains some cost differences, but not all of them. DeepSeek-V4 Flash has the highest cache hit rate (98.5%) and the lowest cost. Yet Claude Opus 4.7 and GPT 5.5 also have cache hit rates near 97%, while their total costs still exceed \$1000. Qwen 3.6-flash has a cache hit rate of 97.6% and costs \$71.47; GLM 5.1 costs \$277.00 at a 96.5% cache hit rate. Cost is therefore jointly affected by model price, input/output tokens, cache policy, and adapter call path. We report cache hit rate as a diagnostic field for cost accounting, not as a measure of model or harness capability.

Claw	Model	Resolved	Pass@1	Cost	Dur	In(M)	Out(M)	Cache
openclaw	GLM 5.1	257	73.4	277.0	586.8	27.6	9.3	96.5
hermes-agent	GLM 5.1	249	71.1	330.6	675.1	93.1	5.5	91.3
zeroclaw	GLM 5.1	246	70.3	383.4	538.2	989.7	4.2	90.4
genericagent	GLM 5.1	221	63.1	85.8	576.4	99.7	5.1	66.8
nanobot	GLM 5.1	213	60.9	768.8	1166.3	333.9	8.5	77.2
openclaw	Qwen 3.6-flash	231	66.0	71.5	636.0	38.9	7.5	97.6
hermes-agent	Qwen 3.6-flash	219	62.6	103.3	638.6	44.3	7.2	97.4
zeroclaw	Qwen 3.6-flash	204	58.3	49.3	428.9	1057.9	6.1	96.9
genericagent	Qwen 3.6-flash	135	38.6	14.5	321.4	103.1	2.8	74.7
nanobot	Qwen 3.6-flash	166	47.4	133.1	562.7	418.8	8.2	63.9

TABLE 3 Claw-axis variation: five claws $\times$ two models on the full 350-instance Claw-SWE-Bench. Cost is total API cost for the full run (USD); In/Out are total input/output tokens (millions); Cache is cache hit rate. Within each model group, the best Pass@1 and lowest Cost are in **bold**.

5.3 Effect of Future-Commit Cleanup

As described in §2.3 and §4, the main experiments clean reachable Git history later than `base_commit` for non-Python SWE-bench-Multilingual instances. To estimate the effect of this treatment on result accounting, we fix OpenClaw and compare nine models before and after cleanup on the 300 Multilingual instances; adapter, prompt, budget, and evaluator settings are otherwise identical.

Figure ?? shows that Pass@1 after cleanup is never higher than before cleanup, consistent with the expectation that future-commit visibility can inflate resolved rate. The impact is not uniform: Claude Opus 4.7 drops the most (84.7% $\rightarrow$ 76.7%, -8.0 pp), Kimi 2.6 drops by 5.0 pp, and Qwen 3.6-flash drops by 2.0 pp; GPT 5.5, MiniMax M2.7, and Seed 2.0-mini change by about 1 pp or less. We therefore use cleanup results as the main accounting basis; the before/after comparison is only used to document the necessity and magnitude of the fairness treatment.

5.4 Variation Along the Claw Axis

To isolate the effect of the claw or harness, we fix the model and sweep the claw axis. Table 3 reports aggregate results for five claws on GLM 5.1 and Qwen 3.6-flash; the cost and cache fields for OpenClaw $\times$ GLM 5.1 use the same leak-fix accounting as Table 2. On GLM 5.1, OpenClaw achieves the highest Pass@1 (73.4%), followed closely by hermes-agent (71.1%) and zeroclaw (70.3%); the generic baseline has the lowest cost (\$85.84) but drops to 63.1% Pass@1. On Qwen 3.6-flash, OpenClaw is again highest (66.0%), with hermes-agent and zeroclaw at 62.6% and 58.3%; the generic baseline falls to 38.6%.

These results show that the claw is not merely a wrapper. With the same GLM 5.1 model, Pass@1 across the five claws ranges from 60.9% to 73.4%, a 12.5 pp spread. With the same Qwen 3.6-flash model, the spread is larger, from 38.6% to 66.0%, or 27.4 pp. In other words, changing only the harness-specific agent loop, tool interface, workspace management, and stopping policy can produce performance differences comparable to, or larger than, neighboring model tiers.

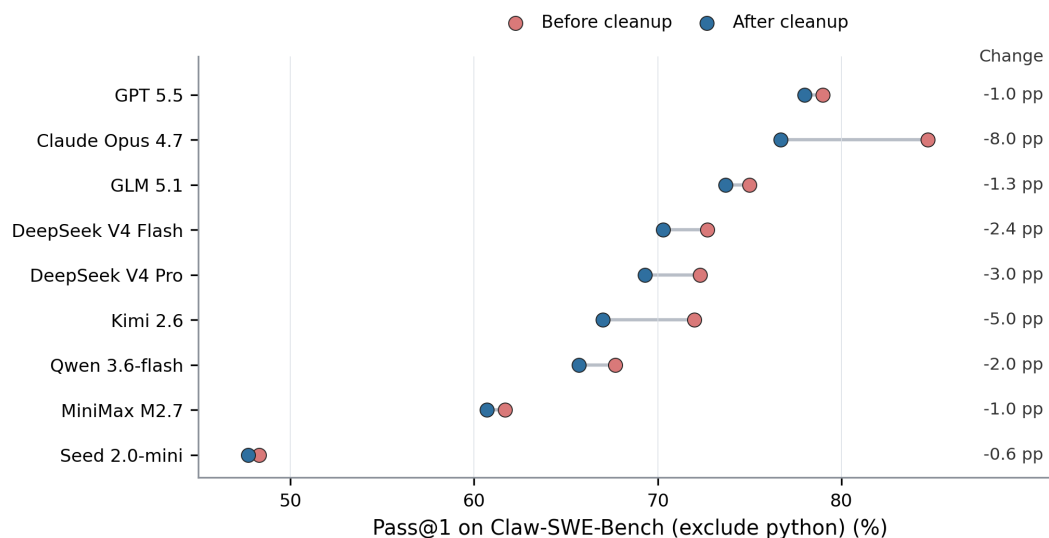


FIGURE 4 Effect of future-commit cleanup on the OpenClaw model sweep. After cleanup, Pass@1 does not increase for any of the nine models; drops range from 0.6 to 8.0 pp.

Cost–accuracy analysis of Figure 1. Cost further changes how claw rankings should be interpreted. Figure 1 is a two-dimensional projection of Table 3: for each claw–model combination, the horizontal axis uses the full 350-instance TOTAL COST, and the vertical axis uses Pass@1 from the same row; OpenClaw × GLM 5.1 follows the leak-fix cost accounting in Table 2. The Pareto frontier consists of points that are not dominated by another combination with both lower cost and higher Pass@1.

In this plane, the lowest-cost endpoint is generic × Qwen 3.6-flash (\$14.50, 38.6%), but its resolved rate is low. Zeroclaw × Qwen 3.6-flash increases cost to \$49.26 and raises Pass@1 to 58.3%; OpenClaw × Qwen 3.6-flash reaches 66.0% at \$71.47. In the GLM 5.1 group, OpenClaw is the high-accuracy endpoint with 73.4% Pass@1 at \$277.00; hermes-agent and zeroclaw have similar resolved rates, but are dominated by OpenClaw × GLM 5.1 because they are both more expensive and less accurate. This result shows that claw comparison cannot be read only as a within-model resolved-rate ranking. A cross-model, cross-claw, cost-aware Pareto view is needed to distinguish genuinely useful operating points from systems that only look close on one axis.

5.5 Interpreting Cache and Cost

Cache hit rate varies substantially across claws. For example, under GLM 5.1, the cache hit rates of OpenClaw, hermes-agent, and zeroclaw are 96.5%, 91.3%, and 90.4%, respectively, while generic is 66.8%. Under Qwen 3.6-flash, OpenClaw, hermes-agent, and zeroclaw are all near 97%, but nanobot is 63.9% and generic is 74.7%. These differences indicate that adapter behavior and provider-side caching can substantially affect the actual API bill. We therefore list cache hit rate in all main result tables so that readers can determine whether cost differences come from model price, token usage, or cache reuse.

At the same time, cache hit rate should not be over-interpreted. A higher cache hit rate does

not necessarily imply higher Pass@1 or a stronger harness; it is first a run-level and billing-level diagnostic. Our conclusions therefore use a two-layer reading: Pass@1 measures the final coding outcome, cost measures the resources required to complete the same evaluation, and cache hit rate explains one important mechanism in cost accounting. Together, these quantities form a repeatable, comparable, and scorable SWE-style coding-agent benchmark.

6 Related Work

Foundational SWE benchmarks. SWE-bench [16] introduced the task formulation we adopt: resolving real GitHub issues against repository-level test suites. Multilingual coverage has since been pushed beyond Python by Multi-SWE-bench / SWE-bench-Multilingual [31], which contributes 300 of our 350 instances, and Python coverage is supplied by the human-validated SWE-bench-Verified-Mini [15], the source of our remaining 50 instances. Subset construction has precedent in SWE-bench Lite [28] and in the more general anchor-set methodology of tinyBenchmarks [25]; our 80-instance Lite subset extends this lineage with a rank-aware ILP. Orthogonal to evaluation, SWE-smith [31] scales SWE training data; its scope is data generation rather than harness comparison.

Single-harness SWE evaluations. A growing line of work proposes individual harnesses and reports their per-system numbers on SWE-bench. SWE-agent [30] introduced the agent-computer-interface scaffold and is the most cited SWE harness. AutoCodeRover [38] adds code-aware retrieval to the agent loop. OpenHands [29] is a generalist agent platform that ships a SWE-bench adapter. mini-SWE-agent [32] represents the minimal-harness end of the design space. SWE-Bench Pro [9] extends the SWE-bench formulation to long-horizon tasks. Each of these reports per-harness resolved rates but does not vary the harness as an experimental axis: prompt, scaffold, runtime, and termination policy are bundled together with each release. Our five-claw $\times$ two-model claw sweep compares heterogeneous harnesses under a fixed outer protocol and reports accuracy together with cost, duration, and cache accounting.

Other coding benchmarks. A broader coding-evaluation literature complements SWE-bench-style issue resolution. HumanEval [5], MBPP [3], and APPS [13] score function-level synthesis; CrossCodeEval [11] targets cross-file completion; CodeClash [33] and PinchBench [18] probe further axes of coding capability. These benchmarks evaluate model code-generation skill rather than harness-mediated agentic behavior on real repositories; we treat them as orthogonal background.

7 Conclusion and Discussion

Conclusion. We introduced CLAW-SWE-BENCH, a 350-instance multilingual SWE-style benchmark, together with CLAW-SWE-BENCH LITE, an 80-instance subset. Through the adapter, a general-purpose agent such as OpenClaw can be constrained to the SWE-bench execution environment, patch contract, and scoring protocol, making it a repeatable, comparable, and scorable coding-agent evaluation target. Lite uses a 17-column cost-aware calibration spanning

both model and harness variation, reproducing full-set aggregate Pass@1 within about 0.4 pp while reducing full-run cost to about 23% of full-350. With prompt, budget, and task set fixed, the OpenClaw $\times$ nine-model sweep and the five-claw $\times$ two-model sweep jointly show that harness choice can reorder system rankings and reshape accuracy–cost trade-offs. SWE-style coding-agent evaluation should therefore report total API cost and cache hit rate alongside Pass@1; the harness should be treated as a first-class controlled variable rather than an implementation detail hidden behind a model score. We hope the full benchmark and Lite subset can serve as stable reference points for follow-on work, allowing new models, harnesses, and adapter changes to be compared under the same tasks, budgets, and cost-accounting conventions while making low-cost debugging and replication easier to incorporate into routine research workflows.

Limitations and future directions. Several boundaries of the current results should be interpreted carefully. First, the main experiments report single-run aggregates; differences of only a few percentage points should therefore not be overinterpreted as stable system superiority, and future work should use multi-seed replication to estimate randomness and run-to-run variance. Second, the claw sweep covers five claws and two representative models, which is sufficient to show that the harness is a first-order variable in SWE-style coding-agent evaluation, but not sufficient to fully decompose harness $\times$ model interactions. A wider model axis would help determine which conclusions arise from general harness mechanisms and which depend on a specific backbone. Third, cost analysis depends on provider-side pricing and cache accounting. We therefore report total API cost, input/output tokens, and cache hit rate together; future releases should also retain raw token traces so that cost differences can be audited and re-priced. More broadly, whether the model–harness non-separability observed here generalizes to web agents or computer-use agents, and how harness components such as agent loop, tool surface, parser, and stopping rule drive accuracy–cost trade-offs, remain open questions.

References

- [1] Alibaba Cloud Qwen Team. Qwen3.6-Flash: Efficient variant of the Qwen3.6 series. Model card and API, <https://github.com/QwenLM/Qwen3.6>, 2026.
- [2] Anthropic. Introducing claude opus 4.7. Product release announcement, <https://www.anthropic.com/news/claude-opus-4-7>, April 2026.
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [4] ByteDance Seed Team. ByteDance Seed 2.0-mini. Model release, <https://seed.bytedance.com/en/seed2>, February 2026.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin,

- Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [6] Claw Bench Team. Claw bench: The definitive ai agent benchmark. GitHub repository, <https://github.com/claw-bench/claw-bench>; leaderboard at <https://clawbench.net>, 2026.
- [7] DeepSeek-AI. DeepSeek-V4-Flash: Fast and economical model of the DeepSeek-V4 preview release. API release notes, <https://api-docs.deepseek.com/news/news260424>; weights at <https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash>, April 2026.
- [8] DeepSeek-AI. DeepSeek-V4 preview release. API release notes, <https://api-docs.deepseek.com/news/news260424>; weights at <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro>, April 2026.
- [9] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- [10] Shuangrui Ding, Xuanlang Dai, Long Xing, Shengyuan Ding, Ziyu Liu, Jingyi Yang, Penghui Yang, Zhixiong Zhang, Xilin Wei, Yubo Ma, Haodong Duan, Jing Shao, Jiaqi Wang, Dahua Lin, Kai Chen, and Yuhang Zang. Wildclawbench: An in-the-wild benchmark for ai agents in the openclaw environment. GitHub repository, <https://github.com/InternLM/WildClawBench>, 2026.
- [11] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.
- [12] Zhiyu Fan, Kirill Vasilevski, Dayi Lin, Boyuan Chen, Yihao Chen, Zhiqing Zhong, Jie M. Zhang, Pinjia He, and Ahmed E. Hassan. Swe-effi: Re-evaluating software ai agent system effectiveness under resource constraints. *arXiv preprint arXiv:2509.09853*, 2025.

- [13] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.
- [14] HKUDS. nanobot: The ultra-lightweight personal AI agent. GitHub repository, <https://github.com/HKUDS/nanobot>, 2026.
- [15] Marius Hobbhahn. Swebench-verified-mini: A compact 50-instance subset of swe-bench verified. GitHub repository, <https://github.com/mariushobbhahn/SWEBench-verified-mini>, 2025. Hugging Face dataset: <https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini>.
- [16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2024.
- [17] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic agent leaderboard: The missing infrastructure for ai agent evaluation. *arXiv preprint arXiv:2510.11977*, 2025.
- [18] Kilo Code Team. Pinchbench: A benchmarking system for evaluating llm models as openclaw coding agents. <https://pinchbench.com/>, 2025. GitHub: <https://github.com/pinchbench>; community-maintained leaderboard, no associated arXiv preprint at time of writing.
- [19] Jiaqing Liang, Jinyi Han, Weijia Li, Xinyi Wang, Zhoujia Zhang, Zishang Jiang, Ying Liao, Tingyun Li, Ying Huang, Hao Shen, Hanyu Wu, Fang Guo, Keyi Wang, Zhonghua Hong, Zhiyu Lu, Lipeng Ma, Sihang Jiang, and Yanghua Xiao. Genericagent: A self-evolving llm agent via contextual information density maximization. Technical report, Shenzhen Aquaintelling and Technology Fudan University, 2026. URL <https://github.com/JinyiHan99/GA-Technical-Report>.
- [20] Fanqing Meng, Lingxiao Du, Zijian Wu, Guanzheng Chen, Xiangyan Liu, Jiaqi Liao, Chonghe Jiang, Zhenglin Wan, Jiawei Gu, Pengfei Zhou, Rui Huang, Ziqi Zhao, Shengyuan Ding, Ailing Yu, Bo Peng, Bowei Xia, Hao Sun, Haotian Liang, Ji Xie, Jiajun Chen, Jiajun Song, Liu Yang, Ming Xu, Qionglin Qiu, Runhao Fu, Shengfang Zhai, Shijian Wang, Tengfei Ma, Tianyi Wu, Weiyang Jin, Yan Wang, Yang Dai, Yao Lai, Youwei Shu, Yue Liu, Yunzhuo Hao, Yuwei Niu, Jinkai Huang, Jiayuan Zhuo, Zhennan Shen, Linyu Wu, Hannah Yao, Charles Chen, Cihang Xie, Yuyin Zhou, Jiaheng Zhang, Zeyu Zheng, Mengkang Hu, and Michael Qizhe Shieh. Clawmark: A living-world benchmark for multi-turn, multi-day, multimodal coworker agents, 2026. URL <https://arxiv.org/abs/2604.23781>.

- [21] MiniMax. MiniMax M2.7: A self-evolving agent model. Model release, <https://www.minimax.io/models/text/m27>, April 2026.
- [22] Moonshot AI. Kimi K2.6: Open-weight 1t-parameter agentic model. Model release, <https://www.kimi.com/ai-models/kimi-k2-6>, April 2026.
- [23] Nous Research. hermes-agent: Self-improving AI agent with built-in learning loop. GitHub repository, <https://github.com/nousresearch/hermes-agent>, 2026.
- [24] OpenAI. Introducing GPT-5.5. Product release announcement, <https://openai.com/index/introducing-gpt-5-5/>, April 2026.
- [25] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinybenchmarks: evaluating llms with fewer examples. *arXiv preprint arXiv:2402.14992*, 2024.
- [26] Peter et al. Steinberger. OpenClaw: Your own personal AI assistant. any OS. any platform. GitHub repository, <https://github.com/openclaw/openclaw>, 2026.
- [27] suyoumo. Clawprobench — a transparent benchmark for true intelligence in real-world ai agents., 2026. URL <https://github.com/suyoumo/ClawProBench>.
- [28] SWE-bench Team. Swe-bench lite: A canonical subset of swe-bench. <https://www.swebench.com/lite.html>, 2024.
- [29] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- [30] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [31] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.
- [32] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Shunyu Yao, Karthik Narasimhan, and Ofir Press. mini-swe-agent: A 100-line minimal ai agent for swe-bench. GitHub repository, <https://github.com/SWE-agent/mini-swe-agent>, 2025.
- [33] John Yang, Kilian Lieret, Joyce Yang, Carlos E. Jimenez, Ofir Press, Ludwig Schmidt, and Diyi Yang. Codeclash: Benchmarking goal-oriented software engineering. *arXiv preprint arXiv:2511.00839*, 2025.

- [34] Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, Qi Liu, Zhifang Sui, and Tong Yang. Claw-eval: Towards trustworthy evaluation of autonomous agents, 2026. URL <https://arxiv.org/abs/2604.06132>.
- [35] Z.ai. GLM-5.1: Open-weight agentic coding model. Model release, <https://huggingface.co/zai-org/GLM-5.1>, April 2026.
- [36] Z.ai. Zclawbench: Evaluating llms as goal-driven agents in openclaw scenarios. Hugging Face dataset, <https://huggingface.co/datasets/zai-org/ZClawBench>, 2026.
- [37] ZeroClaw Labs. zeroclaw: Fast, small, and fully autonomous AI personal assistant infrastructure. GitHub repository, <https://github.com/zeroclaw-labs/zeroclaw>, 2026.
- [38] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. *arXiv preprint arXiv:2404.05427*, 2024.

A Reproducibility Statement

This appendix consolidates the artefacts and protocol required to reproduce every cell of §4.

A.1 Code release

Harness adapters. Five claw-adapter packages – `openclaw_swebench`, `hermes_swebench`, `zeroclaw_swebench`, `nanobot_swebench`, and the `generic` baseline adapter – are released. Each ships `run_infer.py` and `run_eval.py` together with the per-harness orchestrator, agent adapter, and workspace modules. The same registry also hosts the minimal *bare adapter* used for the diagnostic in §5.1.

Lite construction scripts. A Node.js toolkit implements the cost-aware rank-aware Lite selection, K-sweep, sensitivity checks, quartile-stratification logic, and final-report generators.

Figure-generation scripts. `matplotlib` scripts that regenerate the cost–accuracy Pareto figure, the future-commit-cleanup comparison, and the Lite parity panels from the released result workbooks are in the `scripts/` directory of the release (`generate_pareto_figure.py`, `generate_leak_fix_figure.py`, `generate_lite_figures.py`).

A.2 Data release

Claw-SWE-Bench (350 instance IDs plus metadata) and **Claw-SWE-Bench-Lite** (80 instance IDs with cost-aware selection metadata) are released as JSON files. The underlying issues and repositories are hosted on the upstream SWE-bench-Multilingual and SWE-bench-Verified-Mini sources [15, 31]; we redistribute only the curated ID sets and metadata.

A.3 Reproduction protocol

Per-instance run.

```
python3 run_infer.py \  
  --harness <openclaw|hermes|zeroclaw|nanobot|generic> \  
  --dataset multilingual \  
  --model <provider/model_name> \  
  --run_id <run_label> \  
  --timeout 3600 \  
  --workers 3
```

Evaluation.

```
python3 run_eval.py \  
  --predictions artifacts/<run_id>/predictions.jsonl \  
  --dataset_name SWE-bench/SWE-bench_Multilingual \  
  --run_id <run_id> \  
  --max_workers 8
```

The CLI surface is identical across all five claws; per-harness adapter flags are documented in Appendix C.

A.4 Random seeds and runs

A single run per (instance, harness, model) cell is executed with 3-thread concurrency. The 1-repeat choice is a cost-driven trade-off; multi-seed validation on a 50-instance slice is left to future work.

A.5 Compute requirements

Full hardware and cost figures are in Appendix B. In summary: a single 16-core / 61 GiB-RAM Linux server is sufficient, with no GPU required since all model inference is routed through external APIs.

B Compute and Environment Details

This appendix documents the hardware, software, run-time parameters, and aggregate wall-clock cost of the experiments reported in Section 4. All five claws (OPENCLAW, HERMES-AGENT, ZEROCLAW, NANOBOT, and the GENERIC baseline) were executed on a single host with identical run-time parameters; the claw implementation is the only experimental variable.

B.1 Hardware

B.2 Software stack

Item	Value
Architecture	x86_64
OS	Linux 6.8.0-106-generic (Ubuntu)
CPU cores	16
RAM	61 GiB total (~44 GiB available)
Disk	99 GiB volume, ~75 GiB free
GPU	None (model inference is API-only)

TABLE 4 Host hardware. Model inference runs on remote provider APIs, so no local GPU is required; the host is used only for harness orchestration, Docker containers, and patch evaluation.

Component	Version / Path
Docker (eval images)	<code>sweb.eval.x86_64.<instance>:latest</code>
SWE-bench harness	<code>/data/swe-bench-env/</code>
Standalone Python (hermes-agent/nanobot)	CPython 3.12.13 (uv-installed)
hermes virtualenv	<code>/opt/hermes-env/</code> (80 MB)
nanobot virtualenv	<code>/opt/nanobot-env/</code> (211 MB)
openclaw runtime	Node.js 22+; <code>npm install -g openclaw</code>
zeroclaw binary	Rust v0.7.3 (37 MB), bind-mount <code>/usr/local/bin/zeroclaw</code>
generic (GenericAgent) runtime	CPython 3.12 (uv venv), bind-mounted GA install

TABLE 5 Software stack per claw. Standalone Python and the harness virtualenvs are bind-mounted into the SWE-bench evaluation container so that the agent loop runs inside the same container as the patched code.

Parameter	Value
Per-instance timeout	3600 s (60 min)
Repeats per instance	1
Concurrency	3 threads
<code>reasoning_effort</code> (hermes only)	high
Shell tool timeout (each harness)	180 s

TABLE 6 Run-time parameters. These are overridden via CLI flags so that all five claws see the same per-instance budget; the prompt template (Appendix C) is also identical across claws.

B.3 Run-time parameters (held equal across all five claws)

B.4 API providers

Provider	Use
OpenRouter	Default routing (Anthropic, Z-AI/GLM, Qwen, etc.)
DashScope (Alibaba)	Qwen series; some hermes / nanobot runs
Infini-AI	Kimi; some GLM hermes runs
DeepSeek (official)	DeepSeek-V4 Pro / Flash

TABLE 7 API providers used during experiments. Only base URLs and model identifiers are released as part of the artifact; **API keys are NOT included in any released artifact** (see Appendix F).

B.5 Compute cost (aggregate)

The main experiments cover 17 unique (claw, model) columns of 350 instances each: the openclaw $\times$ 9-model sweep (Table 2) plus the 8 non-openclaw cells of the 5-claw $\times$ 2-model sweep (Table 3; the two openclaw cells are shared between the grids). Per-cell mean wall-clock durations are reported in the **Dur** columns of those tables. Summing the per-cell means over instances, the 17 columns account for approximately **1,148 hours** of end-to-end wall-clock (≈ 47.8 days of single-thread execution; ≈ 15.9 days on the 3-thread schedule actually used), of which the model sweep contributes ≈ 671 hours and the non-openclaw claw cells ≈ 477 hours. These figures exclude the bare-adapter diagnostic (§5.1), the pre-cleanup runs of the future-commit comparison (§5.3), and Lite-80 validation runs. Total API cost per column is reported directly in Tables 2 and 3; duration includes remote API latency and is therefore an end-to-end operating measure rather than pure local compute.

C Harness Configurations

All five claws use the IDENTICAL prompt template (D.0), the IDENTICAL run-time parameters (per-instance timeout 3600s, concurrency 3, repeats 1), and the IDENTICAL outer orchestration pattern (build prompt $\rightarrow$ `docker exec` $\rightarrow$ collect `git diff`). The only variation is the inner harness implementation (D.1–D.5): the CLI surface, the agent loop, the tool set, and the model adapter. This is the methodological foundation for the claw sweep in §5.4: prompt and run-time budget are held constant, and the claw becomes the experimental variable.

D.0 Shared prompt template (verbatim)

You are working directly inside a development environment.
The code repository is at /testbed.

IMPORTANT - ENVIRONMENT RULES:

- Do NOT run git add or git commit. Just edit the files and stop.
- Do NOT modify any test files.

Examples:

```
- List files:      ls /testbed/
- Read a file:    cat /testbed/path/to/file
- Search code:    grep -rn "keyword" /testbed/src/
- Edit a file:    sed -i "s/old_text/new_text/g" /testbed/path/to/file
- Run tests:      cd /testbed && <test command>
- Check diff:     cd /testbed && git diff
- Write a script: cat > /tmp/fix.py << "EOF"
import re
# your script here
EOF
python3 /tmp/fix.py
```

Consider the following issue description:

```
<issue_description>
{problem_statement}
</issue_description>
```

Can you help me implement the necessary changes to the repository so that the requirements specified in the <issue_description> are met?

I've already taken care of all changes to any of the test files described in the <issue_description>. This means you DON'T have to modify the testing logic or any of the tests in any way!

The development environment is already set up for you (i.e., all dependencies already installed), so you don't need to install other packages.

Your task is to make the minimal changes to non-test files in the /testbed directory to ensure the <issue_description> is satisfied.

Follow these phases to resolve the issue:

Phase 1. READING: read the problem and reword it in clearer terms

1.1 If there are code or config snippets, express in words

any best practices or conventions in them.

- 1.2 Highlight message errors, method names, variables, file names, stack traces, and technical details.
- 1.3 Explain the problem in clear terms.
- 1.4 Enumerate the steps to reproduce the problem.
- 1.5 Highlight any best practices to take into account when testing and fixing the issue.

Phase 2. RUNNING: figure out how to build and run the tests on the repository

- 2.1 Explore the repo structure to find build scripts, Makefiles, or test configurations.
- 2.2 Try running existing tests to understand the test framework and commands.
- 2.3 If tests fail due to setup, investigate and fix the environment.

Phase 3. EXPLORATION: find the files that are related to the problem and possible solutions

- 3.1 Use grep to search for relevant methods, classes, keywords, and error messages.
- 3.2 Identify all files related to the problem statement.
- 3.3 Propose the methods and files to fix the issue and explain why.
- 3.4 From the possible file locations, select the most likely location to fix the issue.

Phase 4. TEST CREATION: before implementing any fix, create a script to reproduce and verify the issue.

- 4.1 Look at existing test files in the repository to understand the test format/structure.
- 4.2 Create a minimal reproduction script that reproduces the located issue.
- 4.3 Run the reproduction script to confirm you are reproducing the issue.
- 4.4 Adjust the reproduction script as necessary.

Phase 5. FIX ANALYSIS: state clearly the problem and how to fix it

- 5.1 State clearly what the problem is.
- 5.2 State clearly where the problem is located.

- 5.3 State clearly how the test reproduces the issue.
- 5.4 State clearly the best practices to take into account in the fix.
- 5.5 State clearly how to fix the problem.

Phase 6. FIX IMPLEMENTATION: Edit the source code to
implement your chosen solution.

- 6.1 Make minimal, focused changes to fix the issue.

Phase 7. VERIFICATION: Test your implementation thoroughly.

- 7.1 Run your reproduction script to verify the fix works.
- 7.2 Add edge cases to your test script to ensure comprehensive coverage.
- 7.3 Run existing tests related to the modified code to ensure you haven't broken anything.

Phase 8. FINAL REVIEW: Carefully re-read the problem
description and compare your changes
with the base commit `{base_commit}`.

- 8.1 Ensure you've fully addressed all requirements.
- 8.2 Run any tests in the repository related to:
 - 8.2.1 The issue you are fixing
 - 8.2.2 The files you modified
 - 8.2.3 The functions you changed
- 8.3 If any tests fail, revise your implementation until all tests pass.

Be thorough in your exploration, testing, and reasoning.
It's fine if your thinking process is lengthy - quality and completeness are more important than brevity.

This template is rendered with `{problem_statement}` and `{base_commit}` substituted from each instance, then handed to the harness CLI verbatim. It is identical across `openclaw`, `hermes-agent`, `zeroclaw`, and `nanobot`; the generic baseline uses a variant that adds three lines naming `GenericAgent`'s tools and disabling its web tools (see D.5), since `GenericAgent` exposes no config-level tool toggle.

D.1 openclaw

Adapter wrapper. `openclaw` is a stateful Node.js harness. The adapter creates a temporary per-instance `openclaw` agent with its own workspace and session directory, sets a tool deny-list to disable memory, web, session-spawning, sub-agent, cron, and image tools, and invokes the task-solving loop through `openclaw agent` inside the SWE-bench container. `openclaw` emits

structured JSON output, from which the adapter extracts the finish reason, session identifier, and available token-usage metadata. Session JSONL files are backed up before the temporary agent is deleted.

Implementation.Node.js-based agent (entry point `openclaw.mjs`) with full agent lifecycle (create/delete temporary agents per instance), tool deny-list config support, and session backup. It is the only harness with real per-instance agent isolation: each SWE-bench instance gets its own openclaw agent (own workspace, own session store, own memory directory).

Tool inventory.openclaw has a rich tool surface. The harness creates per-instance agents and explicitly *denies* several built-in tools to keep the agent focused on code editing. Tool deny-list is set per-agent by directly editing `~/.openclaw/openclaw.json` (because `openclaw config set` addresses agents by list index).

Status	Tool	Notes
Allowed	<code>read</code>	File read
Allowed	<code>write</code>	File write
Allowed	<code>edit</code>	File edit
Allowed	<code>exec</code>	Shell exec inside container
Allowed	<code>process</code>	Process management
Denied	<code>memory_search</code> , <code>memory_get</code>	No cross-instance memory
Denied	<code>web_search</code> , <code>web_fetch</code>	No web access
Denied	<code>sessions_list</code> , <code>sessions_history</code> , <code>sessions_send</code> , <code>sessions_yield</code> , <code>sessions_spawn</code>	Disabled
Denied	<code>subagents</code>	Disabled
Denied	<code>session_status</code>	Disabled
Denied	<code>cron</code>	Disabled
Denied	<code>image</code>	Disabled

Scaffolding / reasoning loop.

```
For each SWE-bench instance:
  agent_id = f"swe-{instance_id}"

  # Per-instance isolation (unique to openclaw - others are stateless)
  openclaw agents add <agent_id> \
    --workspace /tmp/openclaw-swe-workspaces/<agent_id> \
    --model <model>
  set tools.deny=[memory_*, web_*, sessions_*,
                  subagents, session_status, cron, image]
```

```

on ~/.openclaw/openclaw.json (thread-safe via _config_lock)

start docker container sweb.eval.x86_64.<instance>
prepare_instance(base_commit, setup_gitignore)
prompt = render_template(instance)

docker exec <container> \
    node /usr/lib/node_modules/openclaw/openclaw.mjs \
    agent --agent <agent_id> --message <prompt> \
    --timeout 1200 --json

# openclaw runs ReAct-style reasoning + tool calls
# until LLM returns final answer
# Output is JSON in stdout (gateway mode) or embedded mode

collect_patch via git diff
backup_session:
    copy ~/.openclaw/agents/<agent_id>/sessions/*.jsonl
    -> artifact_dir
openclaw agents delete <agent_id> --force # clean state

```

Stopping conditions.

- **Timeout:** 3600 s (60 min) per instance at run time (CLI override; `config.py` default 1200 s not used). Subprocess wrapper adds 60 s buffer.
- **Finish reasons:** `stop` (success), `error` (non-zero exit / unparseable JSON / status != ok), `empty` (no payloads or all “couldn’t generate”), `timeout`.
- **Retries:** `DEFAULT_MAX_RETRIES = 1`.

Distinctive notes.

- Node.js implementation (lib at `/usr/lib/node_modules/openclaw/openclaw.mjs`).
- Only harness with real per-instance agent isolation: each instance has its own workspace, session store, memory directory.
- Only harness with a tool deny-list: explicitly disables memory/web/sessions/subagents to keep behavior reproducible and fair vs. simpler harnesses.
- Only harness with a structured JSON output protocol: parses `{"status":"ok", "result":{"payloads":[...], "meta":{"...}}}` or embedded mode `{"payloads":[...], "meta":{"...}}`. Other harnesses parse plain stdout.
- Default model: `openrouter/anthropic/claude-opus-4.6`; actual runs override the model per cell through the run-time model flag.

D.2 hermes-agent

Adapter wrapper.hermes is invoked statelessly through a standalone CPython runtime and virtual environment mounted into the container. The adapter calls `hermes chat` with `--yolo`, the shared task prompt, the selected model, and the restricted `terminal,file` toolsets. hermes does not require an agent lifecycle; `create_agent`, `delete_agent`, and `backup_session` are no-ops. The adapter classifies the run outcome from the subprocess exit code, stdout, and timeout status.

Implementation.Python-based agent invoked as a module via uv-installed standalone Python (CPython 3.12.13). CLI invocation is stateless (`--yolo`); `create_agent` / `delete_agent` / `backup_session` are no-ops.

Tool inventory.Set via CLI flag `--toolsets terminal,file`. The two toolsets imply:

Toolset	Tools (typical hermes capabilities)
<code>terminal</code>	shell exec inside the container
<code>file</code>	read / write / edit files

No web, no memory, no sub-agents, no images. hermes’s full tool registry is internal to the `hermes` package and not enumerated in the harness adapter – only the toolset names are passed.

Scaffolding / reasoning loop.

```
For each SWE-bench instance:
  start docker container sweb.eval.x86_64.<instance>
  prepare_instance(base_commit, setup_gitignore)
  prompt = render_template(instance)

  # Stateless invocation - no agent lifecycle
  docker exec <container> \
    -e PYTHONPATH=/opt/hermes-env/lib/python3.12/\
site-packages \
    -e HERMES_HOME=/opt/hermes-config \
    -e OPENROUTER_API_KEY=... \
    -e ANTHROPIC_API_KEY=... \
    /root/.local/share/uv/python/\
  cpython-3.12.13-linux-x86_64-gnu/bin/python3.12 -c '
    import sys
    sys.argv = ["hermes", "chat", "-q", <prompt>,
                "--quiet", "--yolo",
                "--max-turns", "300",
                "--toolsets", "terminal,file",
                "--model", <model>]
```

```

    from hermes_cli.main import main
    sys.exit(main())'

# hermes runs internal reasoning + tool-call loop up to 300 turns
# No JSON output - parses by exit code + stdout presence

collect_patch via git diff

```

Stopping conditions.

- **Timeout:** 3600s (60 min) per instance at run time (CLI override; `config.py` default 1800s not used). Subprocess wrapper adds 120s buffer.
- **Finish reasons** (derived from exit code + stdout): **stop** – exit 0 + non-empty stdout; **empty** – exit 0 + empty stdout; **error** – exit $\neq$ 0; **timeout**.
- **Retries:** `DEFAULT_MAX_RETRIES = 1`.

Distinctive notes.

- No agent lifecycle: each `--yolo` invocation is fully stateless.
- Same wall-clock budget as others: with timeout (3600s) held equal, hermes-specific behavior comes from its CLI surface and toolset, not from a different reasoning budget.
- No structured output: relies on exit code + stdout content for finish-reason classification.
- Run via Python `-c`: avoids shebang path mismatch by importing `hermes_cli.main` directly.
- Default model: `glm-5.1`.
- Implementation: Python (CPython 3.12.13 in `/opt/hermes-env`).

D.3 zeroclaw

Adapter wrapper. `zeroclaw` is a stateless Rust-binary harness. The adapter bind-mounts the `zeroclaw` binary into the container, copies its configuration into a temporary workspace, sets `ZEROCLAW_WORKSPACE`, and invokes `zeroclaw agent` with the shared prompt. This adapter represents a low-dependency single-binary harness design: the same runner lifecycle and patch collector apply even though the internal agent loop is opaque to the benchmark.

Implementation. Single-binary Rust agent (~37 MB, no runtime dependencies). CLI invocation is stateless. Native cost tracking via `costs.jsonl`.

Tool inventory. The adapter does not enumerate tools (tools are compiled into the Rust binary). The agent operates inside `/tmp/zeroclaw-workspace` (set via env `ZEROCLAW_WORKSPACE`), reading and editing files in `/testbed` from within the container. The only externally observable tool surface from the adapter:

Surface	Mechanism
File read/edit	Through Rust binary's internal tools
Shell exec	Through Rust binary's internal tools
Cost / usage logging	<code>costs.jsonl</code> written to <code>/tmp/zeroclaw-workspace/workspace/state/</code>

[Not extracted in source: tool list / signatures (compiled into Rust binary)]

Scaffolding / reasoning loop.

For each SWE-bench instance:

```
start docker container sweb.eval.x86_64.<instance>
prepare_instance(base_commit, setup_gitignore)
prompt = render_template(instance)

docker exec <container> \
  -e ZEROCLAW_WORKSPACE=/tmp/zeroclaw-workspace \
  zeroclaw agent -m <prompt>

# Rust binary runs internal agent loop up to 50 turns
# Cost data written to
#   /tmp/zeroclaw-workspace/workspace/state/costs.jsonl
# On each turn: input/output/total token counts logged

copy /tmp/zeroclaw-workspace/workspace/state/costs.jsonl
  -> artifact_dir/costs.jsonl
collect_patch via git diff
```

After agent run, the orchestrator parses `costs.jsonl` to extract per-instance metrics: `turns` (count of cost lines = turns), and `input_tokens`, `output_tokens`, `total_tokens` summed across turns.

Stopping conditions.

- **Timeout:** 3600s (60 min) per instance at run time (CLI override; `config.py` default 1200s not used). Subprocess wrapper adds 120s buffer.
- **Finish reasons** (derived from exit code + stdout): `stop` – exit 0 + non-empty stdout; `empty` – exit 0 + empty stdout; `error` – exit $\neq$ 0; `timeout`.
- **Retries:** `DEFAULT_MAX_RETRIES = 1`.

Distinctive notes.

- Only harness implemented in Rust: single 37 MB binary at `/usr/local/bin/zeroclaw`. No Python venv, no Node modules.

- Only harness with native per-turn cost logging: `costs.jsonl` records `{turn, usage: {input_tokens, output_tokens, total_tokens}}` per turn. Useful for the cost analysis in §5.5.
- No agent lifecycle: stateless invocation. Workspace is `/tmp/zeroclaw-workspace` (shared per container instance, not per-instance).
- Default model: `glm-5.1`.
- Implementation: Rust binary, bind-mounted into container.
- *[Not extracted in source: `internal reasoning loop type (ReAct? Reflection?)`].*

D.4 nanobot

Adapter wrapper. `nanobot` is a stateless Python harness that runs with `/testbed` as its workspace. During execution, `nanobot` creates workspace metadata files such as `AGENTS.md`, `SOUL.md`, `TOOLS.md`, `USER.md`, `memory/`, and `sessions/`. The adapter therefore copies the session JSONL log out of the container and then removes these metadata files before patch collection, ensuring that the final diff reflects source-code edits rather than harness bookkeeping.

Implementation. Python-based agent invoked via uv-installed standalone Python (CPython 3.12.13). The harness creates filesystem metadata files (`AGENTS.md`, `SOUL.md`, etc.) in the workspace which the harness scrubs before patch collection.

Tool inventory. The adapter does not enumerate tools (tools come from the `nanobot` package internals; configured via `/opt/nanobot-config/config.json`). External observation:

Surface	Mechanism
File read/edit	Within <code>-w /testbed</code> workspace
Shell exec	Internal <code>nanobot</code> tools
Session log	<code>/testbed/sessions/cli_direct.jsonl</code> (full conversation w/ tool calls)
Metadata files	<code>AGENTS.md</code> , <code>HEARTBEAT.md</code> , <code>SOUL.md</code> , <code>TOOLS.md</code> , <code>USER.md</code> , <code>memory/</code> , <code>sessions/</code> in <code>/testbed</code> (pollution)

Output flags: `--no-markdown` (suppresses markdown formatting in stdout), `--no-logs` (suppresses verbose logging). *[Not extracted in source: `contents of /opt/nanobot-config/config.json`].*

Scaffolding / reasoning loop.

For each SWE-bench instance:

```
start docker container sweb.eval.x86_64.<instance>
```

```

prepare_instance(base_commit, setup_gitignore)
prompt = render_template(instance)

docker exec <container> \
    -e PYTHONPATH=/opt/nanobot-env/lib/python3.12/\
site-packages \
    /root/.local/share/uv/python/\
cpython-3.12.13-linux-x86_64-gnu/bin/python3.12 -c '
    import sys
    sys.argv = ["nanobot", "agent", "-m", <prompt>,
                "-c", "/opt/nanobot-config/config.json",
                "-w", "/testbed",
                "--no-markdown", "--no-logs"]
    from nanobot.cli.commands import app
    app()'

# nanobot runs internal agent loop up to 30 turns
# Session log auto-saved to
#   /testbed/sessions/cli_direct.jsonl

# AFTER run, BEFORE patch collection:
copy /testbed/sessions/cli_direct.jsonl
    -> artifact_dir/session.jsonl
docker exec rm -rf \
    /testbed/{AGENTS,HEARTBEAT,SOUL,TOOLS,USER}.md \
    /testbed/memory/ /testbed/sessions/    # scrub

collect_patch via git diff    # now clean

```

Stopping conditions.

- **Timeout:** 3600s (60 min) per instance at run time (CLI override; `config.py` default 1200s not used). Subprocess wrapper adds 120s buffer.
- **Finish reasons** (derived from exit code + stdout): **stop** – exit 0 + non-empty stdout; **empty** – exit 0 + empty stdout; **error** – exit $\neq$ 0; **timeout**.
- **Retries:** `DEFAULT_MAX_RETRIES = 1`.

Distinctive notes.

- Workspace pollution + scrub is the main nanobot-specific operational concern.
- nanobot writes its own metadata files (`AGENTS.md`, `SOUL.md`, `TOOLS.md`, `USER.md`, `memory/`, `sessions/`) into the workspace `/testbed`. The harness deletes these *before* `git diff` to keep the patch clean. Other harnesses do not have this concern.

- Session log preservation: harness copies the full conversation JSONL out before scrub – this gives the richest per-turn audit trail (better than zeroclaw’s `costs.jsonl` which only has token counts).
- Model in config, not CLI: nanobot has no `--model` flag; model is configured in `/opt/nanobot-config/config.json`. Default = `glm-5.1`.
- Implementation: Python (CPython 3.12.13 in `/opt/nanobot-env`).

D.5 generic (GenericAgent)

Role. The generic baseline is the fifth claw of the claw sweep (§5.4). It wraps the open-source `lsdefine/GenericAgent` project, is registered in the harness map under the string ID `generic`, and runs under the same adapter protocol, shared prompt template (D.0), and outer wall-clock budget as the other four claws. It is distinct from the *bare adapter* diagnostic of §5.1: the generic claw edits files in `/testbed` and has its patch exported from Git state by the runner, whereas the bare adapter asks the model to emit a unified diff directly in its final response.

Adapter wrapper. `GenericAgent` provides a headless task mode in which the agent reads `temp/<id>/input.txt`, runs its agent loop, and writes `temp/<id>/output.txt` terminated by a literal `[ROUND END]` sentinel. The adapter bind-mounts the host `GenericAgent` install, its uv-managed CPython 3.12 virtualenv, and a per-instance writable temp directory into the SWE-bench container; pre-writes `input.txt` with the rendered task prompt; launches the agent via `docker exec` with working directory `/testbed` (`agentmain.py --task <id> --llm_no <n> --nobg --verbose`); and polls `output.txt` every 2 seconds for the sentinel. The patch is collected runner-side via `git diff` against the base commit, identical to the other claws. Provider and model are selected by the `--llm_no` index into a key-configuration file rather than a `--model` flag.

Tool inventory. `GenericAgent` ships a fixed function-calling tool schema: `code_run`, `file_read`, `file_patch`, `file_write`, `web_scan`, `web_execute_js`, `update_working_checkpoint`, `ask_user`, and `start_long_term_update`. The harness exposes no CLI flag for disabling individual tools, so the web tools (`web_scan`, `web_execute_js`) and network access are disabled by prompt instruction, with no source patches to `GenericAgent` itself.

Stopping conditions.

- **Timeout:** 3600 s (60 min) per instance at run time (CLI override; the adapter’s 1800 s default is not used). Subprocess wrapper adds a 120 s buffer.
- **Finish reasons:** `stop` – `[ROUND END]` sentinel observed; `timeout` – deadline reached without the sentinel; `error` – the process exits without producing the sentinel.
- **Retries:** `DEFAULT_MAX_RETRIES = 1`.

Distinctive notes.

- Only claw driven through a file-based input/output contract (`input.txt` / `output.txt` with

sentinel polling) rather than a CLI conversation or structured JSON output.

- Token usage (input / output / cache-read / cache-write) is parsed from the agent’s stdout accounting lines and stored in per-instance `metadata.json`; DashScope-routed runs go through a local cache-accounting proxy.
- Implementation: Python (uv-managed CPython 3.12 virtualenv), function-calling agent loop.

D Lite-80 Construction Detail

This appendix records the released Lite-80 construction used in Section 3. The current release is the cost-aware 17-column version built from the LeakFix combined-350 data and the 5-claw cross-harness grid. It supersedes the earlier resolve-only variant.

D.1 Calibration Pool and Constraints

Lite selection is calibrated on 17 evaluation columns: 9 openclaw model columns plus 8 non-openclaw cross-claw columns. The latter are the four additional claws (HERMES, NANOBOT, ZEROCLAW, GENERIC) evaluated on the two shared models, GLM 5.1 and Qwen 3.6-flash. The ranking and cost gates also use the corresponding 5-claw $\times$ 2-model universe when checking within-model claw comparisons.

For each language, the released subset selects exactly 10 instances. Within a language, the 10 selected instances must follow the fixed 2/3/3/2 allocation over difficulty quartiles $Q_1/Q_2/Q_3/Q_4$. Quartiles are computed from the mean resolved rate across the calibration pool, so the strata reflect multi-model and multi-claw difficulty rather than a single harness’s behavior.

D.2 Objective and Solver

Let $x_i \in \{0, 1\}$ indicate whether instance i is selected. The selection loss combines three terms. First, a resolve-rate L1 term matches Lite-implied and full-350 rates over the 17×8 grid of calibration column by language. Second, a pairwise ranking hinge penalizes column inversions: for column pairs whose full-set rates differ by more than `RANK_EPS` = 0.03, the loss is active when the Lite-predicted ordering is wrong or within a margin of 0.05. The released setting uses $\lambda = 1.0$. Third, a cost term with `cost_alpha` = 1 matches Lite and full costs in log space, so the subset preserves the operating-cost structure rather than only the resolve-rate structure.

The constrained search is run independently per language with 200 random restarts followed by same-quartile 1-swap local search. Because all swaps stay inside the same quartile, every candidate subset remains feasible with respect to both language count and difficulty allocation.

D.3 K-sweep Decision

The released size is selected by sweeping K instances per language and checking resolve, cost, and operational gates under sensitivity scenarios. Table 8 shows the resulting minimum passing K values. The band is $K^* \in [8, 10]$; the release chooses the conservative maximum $K = 10$, yielding

$8 \times 10 = 80$ instances. At this point all resolve gates (R-A/R-B/R-C), cost gates (C-A/C-B/C-C), and the operational composite gate pass.

Scenario	Driver class	K_{scenario}^*
main	—	9
MARGIN_0	margin	10
MARGIN_0.5overK	margin	10
restart_50	restart	10
restart_500	restart	8
seed_offset_1000	seed	8
seed_offset_2000	seed	10
mirror_B	mirror parity	9
mirror_A	mirror parity	9
release	K_{max}^*	10

TABLE 8 Sensitivity envelope for the Lite size decision. Each row reports the smallest passing K under one perturbation scenario.

D.4 Distribution and Cross-claw Validation

Table 9 reports the per-language distribution match. Averaged over all 17 calibration columns, full-350 has Pass@1 0.639 and Lite-80 has Pass@1 0.643, a +0.4 percentage-point difference.

Language	Full count	Lite count	Full rate	Lite rate	Δ
Java	43	10	0.699	0.694	−0.005
Go	42	10	0.476	0.476	+0.000
Rust	43	10	0.748	0.741	−0.007
JS/TS	43	10	0.616	0.612	−0.004
C/C++	42	10	0.647	0.676	+0.029
Ruby	44	10	0.603	0.629	+0.027
PHP	43	10	0.647	0.641	−0.006
Python	50	10	0.666	0.671	+0.005
Total	350	80	0.639	0.643	+0.004

TABLE 9 Per-language distribution match for the released cost-aware Lite-80. Rates are unweighted averages over the 17 calibration columns.

Table 10 reports the direct cross-claw parity check on the 5-claw $\times$ 2-model grid. The mean absolute Lite-vs-full gap is 1.88 percentage points, and the maximum gap is 3.68 percentage

points.

Claw	Model	Full rate	Lite rate	Δ
openclaw	GLM 5.1	0.734	0.750	+0.016
openclaw	Qwen 3.6-flash	0.660	0.688	+0.028
hermes	GLM 5.1	0.711	0.738	+0.026
hermes	Qwen 3.6-flash	0.626	0.613	−0.013
zeroclaw	GLM 5.1	0.703	0.725	+0.022
zeroclaw	Qwen 3.6-flash	0.583	0.575	−0.008
nanobot	GLM 5.1	0.609	0.600	−0.009
nanobot	Qwen 3.6-flash	0.474	0.438	−0.037
generic	GLM 5.1	0.631	0.625	−0.006
generic	Qwen 3.6-flash	0.386	0.362	−0.023

TABLE 10 Cross-claw parity on the two shared models. Δ is Lite-80 Pass@1 minus full-350 Pass@1.

D.5 Coverage and Cost

The released Lite-80 contains 34 unique repositories, covering $34/43 = 79\%$ of the repositories in the full benchmark. Its full-run resource ratio is close to the raw instance ratio: true cost is about 22.9% of full-350, input tokens about 22.2%, output tokens about 23.6%, cache-read tokens about 22.6%, and wall-clock duration about 23.0%. This supports the intended use of Lite as an approximately four-times cheaper evaluation surface for debugging, regression testing, and preliminary model or claw comparisons.

E Per-Language Breakdown

This appendix expands on §5 with the full per-language breakdown tables that did not fit in the main text. All numbers are computed directly from the released result workbooks (the leak-fix combined-350 model-sweep report and the cache-fixed 5-claw cross report; see §A); no values are estimated. All Multilingual results use the future-commit cleanup setting of §5.3.

E.1 Per-Language Resolved Rate (openclaw $\times$ 9 models)

Table 11 reports resolve rate on the 350-instance benchmark, disaggregated by language, for the 9 models evaluated under the openclaw harness (the model sweep across LLMs, §5.2). Models are listed in descending order of overall resolve rate; totals match Table 2 in the main text.

Commentary.Java and Rust dominate the per-model maxima: Rust is the best language for 5 of 9 models, Java for 3, and the two tie for DeepSeek-V4 Pro. Go is the worst language for all 9 models without exception (range 33.3–61.9%), sitting 11–22 pp below each model’s overall

Model	Java	Go	Rust	JS/TS	C/C++	Ruby	PHP	Python	Total
GPT 5.5	86.0	<u>61.9</u>	93.0	79.1	81.0	70.5	74.4	78.0	78.0
Claude Opus 4.7	86.0	<u>61.9</u>	88.4	81.4	71.4	68.2	76.7	82.0	77.1
GLM 5.1	76.7	<u>57.1</u>	86.0	74.4	81.0	68.2	72.1	72.0	73.4
DeepSeek-V4 Pro	79.1	<u>50.0</u>	79.1	74.4	73.8	72.7	76.7	68.0	71.7
DeepSeek-V4 Flash	81.4	<u>50.0</u>	79.1	74.4	73.8	63.6	69.8	70.0	70.3
Kimi 2.6	69.8	<u>50.0</u>	74.4	65.1	71.4	68.2	69.8	66.0	66.9
Qwen 3.6-flash	83.7	<u>54.8</u>	74.4	58.1	66.7	59.1	62.8	68.0	66.0
MiniMax M2.7	65.1	<u>47.6</u>	76.7	55.8	61.9	61.4	55.8	66.0	61.4
Seed 2.0-mini	60.5	<u>33.3</u>	55.8	44.2	54.8	36.4	48.8	54.0	48.6

TABLE 11 Per-language resolved rate (%) under the openclaw harness for each of the 9 models, leak-fix accounting. Best language per model in **bold**, worst underlined. The Total column matches Table 2.

mean. The two leaders differ in profile: GPT 5.5 peaks sharply on Rust (93.0%), while Claude Opus 4.7 carries the column maxima for JS/TS (81.4%) and Python (82.0%). Qwen 3.6-flash shows the most Java-skewed profile of the sweep (83.7% Java against a 66.0% overall mean).

E.2 Per-Language Resolved Rate (claw sweep, 5 claws $\times$ 2 models)

Table 12 reports the analogous breakdown for the claw sweep (§5.4): five claws on GLM 5.1 and Qwen 3.6-flash, 10 cells in total. Each row is one (claw, model) cell; columns are the 8 languages plus overall. Totals match Table 3.

Claw	Model	Java	Go	Rust	JS/TS	C/C++	Ruby	PHP	Python	Total
openclaw	GLM 5.1	76.7	<u>57.1</u>	86.0	74.4	81.0	68.2	72.1	72.0	73.4
hermes	GLM 5.1	72.1	<u>54.8</u>	81.4	72.1	73.8	72.7	72.1	70.0	71.1
zeroclaw	GLM 5.1	83.7	<u>50.0</u>	86.0	69.8	69.0	65.9	69.8	68.0	70.3
generic	GLM 5.1	65.1	<u>47.6</u>	69.8	55.8	66.7	63.6	65.1	70.0	63.1
nanobot	GLM 5.1	65.1	<u>38.1</u>	67.4	58.1	61.9	61.4	67.4	66.0	60.9
openclaw	Qwen 3.6-flash	83.7	<u>54.8</u>	74.4	58.1	66.7	59.1	62.8	68.0	66.0
hermes	Qwen 3.6-flash	65.1	<u>42.9</u>	74.4	58.1	61.9	65.9	65.1	66.0	62.6
zeroclaw	Qwen 3.6-flash	62.8	<u>47.6</u>	72.1	55.8	<u>47.6</u>	56.8	55.8	66.0	58.3
nanobot	Qwen 3.6-flash	41.9	42.9	58.1	<u>37.2</u>	45.2	45.5	48.8	58.0	47.4
generic	Qwen 3.6-flash	44.2	<u>19.0</u>	55.8	32.6	38.1	25.0	48.8	44.0	38.6

TABLE 12 Per-language resolved rate (%) for each of the 10 (claw, model) cells in the claw sweep. **Bold** = best language in row; underline = worst.

Commentary. Go remains the hardest language in 8 of 10 cells; the exceptions are nanobot $\times$ Qwen 3.6-flash, whose weakest language is JS/TS (37.2%), and zeroclaw $\times$ Qwen 3.6-flash, where Go ties C/C++ (47.6%). The within-model claw spread is strongly language-dependent and widens on the small model: on GLM 5.1 the largest per-language claw spread is 19.0 pp (Go, 38.1–57.1%), whereas on Qwen 3.6-flash it reaches 41.8 pp on Java (41.9–83.7%), 40.9 pp on Ruby (25.0–65.9%), and 35.8 pp on Go (19.0–54.8%). The generic baseline collapses hardest on Go (19.0%) and Ruby (25.0%) under Qwen 3.6-flash, consistent with the aggregate 27.4 pp spread reported in §5.4. zeroclaw $\times$ GLM 5.1 posts the best Java cell of the grid (83.7%) despite its mid-pack overall rate, showing that claw rankings can invert across languages even within one model.

F License and Ethics

License of the released artifact. This benchmark is derived from two upstream sources, both released under MIT. (i) **SWE-bench Multilingual** (Khandpur, Lieret, Jimenez, Press, Yang; [31]) is hosted at huggingface.co/datasets/SWE-bench/SWE-bench_Multilingual, released as part of the official SWE-bench project. (ii) **SWE-bench Verified-Mini** (Hobbhahn; [15]) is an MIT-licensed subset of SWE-bench Verified (the OpenAI-validated 500-instance Python subset), hosted at github.com/mariushobbhahn/SWEBench-verified-mini.

We retain both upstream LICENSE files and citations. The underlying source code in each task instance retains the license of its original GitHub repository; both upstream datasets aggregate real-world repositories with heterogeneous licenses, including BSD (Django, Flask, sympy, astropy, sphinx), Apache 2.0 (requests, xarray, caddyserver/caddy), MIT, and a small number of GPL-licensed projects (notably pylint, GPL-2). Users redistributing patches or derivative work must comply with the per-repository license. `REPO_LICENSES.md` in the released artifact lists every underlying repository and its current upstream license.

Broader impacts and ethical considerations. Claw-SWE-Bench measures coding-agent capability on real software bugs. This dual-use surface mirrors that of upstream SWE-bench: stronger coding-agent performance benefits software maintenance and accessibility, but the same capability can in principle be applied to autonomous exploitation of vulnerable software. We mitigate by releasing only the benchmark protocol and the instance `instance_id` list (not vulnerable patches as targets), inheriting the upstream curation that excludes security-sensitive issues.